



Explanation

The core idea

Just like your `threesum` calls `twosum`, and `fourSum` calls `threesum` — this is the same nesting, but written so the "depth" is a parameter `k` instead of a hardcoded function name. Each call peels off **one number** and asks: "find `k-1` more numbers, starting after me, that sum to `target - nums[i]`."

So `4Sum` → `kSum(..., k=4)` → loop picks `nums[i]`, calls `kSum(..., target-nums[i], k=3)` → that loop picks another number, calls `kSum(..., k=2)` → base case, `twoSum`.

This is structurally identical to:

```
fourSum picks nums[i] → calls threesum(target - nums[i])
threesum picks nums[i] → calls twosum(target - nums[i])
```

except now it's one function calling itself instead of three different functions.

Line by line

1. Early exit / pruning:

```
if start == n or nums[start] * k > target or nums[-1] * k < target:
    return res
```

- `start == n`: no elements left to pick from.
- `nums[start] * k > target`: since the array is sorted, `nums[start]` is the **smallest** remaining value. If even the smallest value repeated `k` times already exceeds target, no combination from here on can work — stop.

- `nums[-1] * k < target`: `nums[-1]` is the **largest** value overall. If even the largest value repeated `k` times can't reach the target, no combination works either.

This is the generalized version of `if i == 0 or nums[i] != nums[i-1]` style pruning you had — just extended to sums instead of just duplicates.

2. Base case:

```
if k == 2:
    return self.twoSum(nums, target, start)
```

This is exactly your `twosum`, just parameterized to start from index `start` instead of `i+1` hardcoded.

3. The loop (this replaces your `threesum`'s loop and `fourSum`'s loop, unified):

```
for i in range(start, n - k + 1):
```

- We loop from `start` to `n - k`. The `n - k + 1` upper bound ensures there are always enough elements left after `i` to form the remaining `k-1` numbers (no point starting too late).

4. Duplicate skip (same idea as your `if i == idx+1 or nums[i] != nums[i-1]`):

```
if i > start and nums[i] == nums[i - 1]:
    continue
```

Skip over repeated values for *this position* so we don't generate duplicate quadruplets/quintuplets/etc. `i > start` (not `i > 0`) because we only compare against the previous element **within this recursive call's window** — comparing to something before `start` would be comparing against an element already locked in by a previous pick.

5. More pruning, same logic as step 1 but applied per-iteration:

```

if nums[i] * k > target:
    break
if nums[i] + nums[-1] * (k - 1) < target:
    continue

```

- If `nums[i]` itself is already too big to be the smallest of `k` numbers summing to target, since array is sorted, everything after is even bigger — `break` out entirely.
- If `nums[i]` plus the max possible sum of the remaining `k-1` numbers (all taking the largest value `nums[-1]`) is still less than target, this `nums[i]` is too small — skip and try the next one (`continue`, not `break`, since a larger `nums[i]` might work).

6. The recursive call — this is the heart of it:

```

for sub in self.kSum(nums, target - nums[i], i + 1, k - 1):
    res.append([nums[i]] + sub)

```

- We've fixed `nums[i]` as one of our `k` numbers.
- We ask: "give me all unique `(k-1)` tuples starting from index `i+1` that sum to `target - nums[i]`."
- Whatever tuples come back, we prepend `nums[i]` to each and add to our results.

This is **identical** to what your `fourSum` does:

```

diff = target - nums[i]
self.threesum(nums[i], i, nums, diff, n)

```

except instead of calling a specifically-named function (`threesum`), it calls itself with `k-1`. That's the whole trick — recursion replaces the chain of differently-named functions.

Visualizing the call stack for 4Sum, target=0, nums=[-2,-1,0,0,1,2]

```
kSum(k=4, start=0, target=0)
├ i=0, nums[i]=-2 → kSum(k=3, start=1, target=2)
│   ├── i=1, nums[i]=-1 → kSum(k=2, start=2, target=3)
│   │   └ twoSum(start=2, target=3) → [[1,2]]
│   │   → combine: [-1,1,2]
│   │   → combine with outer -2: [-2,-1,1,2] ✓
│   ├── i=2, nums[i]=0 → kSum(k=2, start=3, target=2)
│   │   └ twoSum → [[0,2]]
│   │   → [-2,0,0,2] ✓
│   └ ...
├ i=1, nums[i]=-1 → kSum(k=3, start=2, target=1)
└ ... eventually finds [-1,0,0,1] ✓
└ ...
```

Notice it's the exact same "fix one number, recurse on the rest with a smaller k and adjusted target" pattern you already wrote three times — just collapsed into one self-similar function.

Let's trace it with a small, concrete example: `nums = [1, 0, -1, 0, -2, 2]`, `target = 0`, `k = 4` (your Example 1).

Step 0: Sort

```
nums = [-2, -1, 0, 0, 1, 2]
indices: 0  1  2  3  4  5
```

Call 1: `kSum(nums, target=0, start=0, k=4)`

Pruning check: `nums[0]*4 = -8 > 0` ? No. `nums[-1]*4 = 8 < 0` ? No. Continue.

`k != 2`, so we loop `i` from `0` to `n-k = 2` (inclusive), i.e. `i = 0, 1, 2`.

`i = 0` → `nums[0] = -2`

- Not a duplicate (`i == start`).

- Pruning: $2*4 = -8 > 0$? No. $2 + 2*3 = 4 < 0$? No. Continue.
- Recurse: `kSum(nums, target=0-(-2)=2, start=1, k=3)` **Call 2:** `kSum(target=2, start=1, k=3)`
 Loop `i` from `1` to `n-k=3`, i.e. `i = 1, 2, 3`.

Call 2 returns `[[-1,1,2], [0,0,2]]`

- **i = 1** → `nums[1] = -1`
 - Recurse: `kSum(target=2-(-1)=3, start=2, k=2)` **Call 3 (base case):**
`kSum(target=3, start=2, k=2)` → `twoSum(nums, target=3, start=2)`
 Two-pointer on `nums[2:] = [0,0,1,2]`, looking for pairs summing to 3:

Returns `[[1,2]]`

- `j=2(0), k=5(2)` → `sum=2 < 3` → `j++`
- `j=3(0), k=5(2)` → `sum=2 < 3` → `j++`
- `j=4(1), k=5(2)` → `sum=3` ✓ → record `[1,2]`

- Back in Call 2: `sub = [1,2]` → `res.append([-1] + [1,2])` = `[-1,1,2]`

- **i = 2** → `nums[2] = 0`
 - Pruning: $0 + 2*2=4 < 2$? No, continue normally.
 - Recurse: `kSum(target=2-0=2, start=3, k=2)` → `twoSum(target=2, start=3)`
 Two-pointer on `nums[3:] = [0,1,2]`:

Returns `[[0,2]]`

- `j=3(0), k=5(2)` → `sum=2` ✓ → record `[0,2]`
- `j++, k--` → `j=4, k=4` → loop ends (`j` not `<` `k`)

- Back in Call 2: `sub=[0,2]` → `res.append([0]+[0,2])` = `[0,0,2]`

- **i = 3** → `nums[3] = 0` — but `nums[3] == nums[2]` and `3 > start(1)` → **skip (duplicate)**

- Back in Call 1 (`i=0`): combine `2` with each sub-result:

- `[-2] + [-1,1,2]` = `[-2,-1,1,2]` ✓

◦ `[-2] + [0,0,2] = [-2,0,0,2]` ✓

i = 1 → `nums[1] = -1`

- Recurse: `kSum(target=0-(-1)=1, start=2, k=3)` **Call 2':** `kSum(target=1, start=2, k=3)`
Loop `i` from `2` to `3`.

Call 2' returns `[[0,0,1]]`

◦ **i = 2** → `nums[2] = 0`

- Recurse: `kSum(target=1-0=1, start=3, k=2)` → `twoSum(target=1, start=3)`
Two-pointer on `nums[3:] = [0,1,2]`:

Returns `[[0,1]]`

- `j=3(0), k=5(2)` → `sum=2 > 1` → `k--`
- `j=3(0), k=4(1)` → `sum=1` ✓ → record `[0,1]`

- `sub=[0,1]` → `res.append([0]+[0,1]) = [0,0,1]`

◦ **i = 3** → `nums[3]=0`, duplicate of `nums[2]` and `3 > start(2)` → **skip**

- Back in Call 1 (i=1): `[-1] + [0,0,1] = [-1,0,0,1]` ✓

i = 2 → `nums[2] = 0`

- Pruning check: `nums[i] + nums[-1]*(k-1) = 0 + 2*3 = 6 < 0` ? No.
- Recurse: `kSum(target=0-0=0, start=3, k=3)`
Loop `i` from `3` to `4`.

Returns `[]`

- **i=3** → `nums[3]=0` → recurse `kSum(target=0, start=4, k=2)` → `twoSum` on `nums[4:]=[1,2]` → `sum=3≠0`, no match → `[]`
- **i=4** → `nums[4]=1` → recurse `kSum(target=-1, start=5, k=2)` → `twoSum` on `[2]` → only one element, loop `j<k` never true → `[]`
- No results added.

Final result from Call 1:

```
[[ -2, -1, 1, 2], [ -2, 0, 0, 2], [ -1, 0, 0, 1]]
```

This matches the expected output exactly. ✓

The pattern to notice

Every recursive call does the **same three things**:

1. Try pruning early (sorted array means min/max bounds are easy to check).
2. Loop through candidates for "the next number," skipping duplicates.
3. Recurse with `k-1` and a reduced target, then glue the chosen number onto whatever the recursion returns.

It bottoms out at `k=2` where two-pointer is more efficient than recursing further (since for `k=2` you don't need to "choose and recurse" — you can do it in one $O(n)$ sweep instead of $O(n^2)$).

That's really it — it's your `fourSum → threesum → twosum` chain, but instead of three separate functions, one function calls a smaller version of itself until it hits the two-pointer base case.
